DOOMSDAY ALGORITHM — JOURNEY SPECIFICATION

The cinematic flow: landing → incursion → arrival → four rounds → transmission complete

Companion to `UI_DESIGN_SPEC.md`. That document defines *what things look like*. This one defines *what happens, in what order, and why*.

---

0. READ THIS FIRST

0.1 RELATIONSHIP TO THE OTHER SPECS

Document	Owns
`FINAL_EVENT_PLAN.md`	Rules, scoring, judging, organizer run sheet
`UI_DESIGN_SPEC.md`	Tokens, components, per-page layout, the DOM contract
`JOURNEY_SPEC.md` (this)	**Screen sequence, transitions, state machine, the globe, interstitials**

This document **extends** `UI_DESIGN_SPEC.md`. Every token, component, and DOM-contract rule there still applies. Where this document introduces a new visual mode (§2), it says so explicitly and defines it fully.

0.2 TIMELINE RISK — THE HONEST VERSION

The event runs **September 11, 2026**. This document specifies a substantially more ambitious front end than what is currently deployed and verified working.

The currently deployed site works. It is tested end-to-end against real Firebase. Everything in this document is additive polish on top of a functioning game.

Therefore: **§11 defines a fallback ladder.** Build in that order. At any point where time runs out, the level you have reached must be shippable on its own. Do not begin a level you cannot finish.

Above all: **§10.4 specifies a kill switch.** If the cinematic intro fails to load for any reason — a slow laptop, a blocked CDN, a JS error — the participant lands on a plain functional page and plays the game. The intro is never allowed to be a single point of failure for the event.

0.3 THE DATASET — ALREADY DONE

Asked for first, and already complete. Generated with seed `42`, deployed at `/data/`, verified. Teams receive:

File	Rows	Size	Contents
`films.csv`	28	1.6 KB	phase, year, budget_m, opening_weekend_m, worldwide_gross_m, critic_score, audience_score
`characters.csv`	60	1.5 KB	name, faction, first_film, powered
`appearances.csv`	280	15 KB	character, film, screentime_min, dialogue_lines, billing_order, final_billing_position, survived
`co_appearances.csv`	1,075	52 KB	character_a, character_b, film, shared_scenes
`post_credits.csv`	27	1.7 KB	film, character_teased, paid_off_in_film
`roster.csv`	60	1.5 KB	name, faction, powered
`questions.json`	34	6 KB	10 Explore + 24 Predict, no answers
`draft_pool.json`	43	0.7 KB	draftable character names

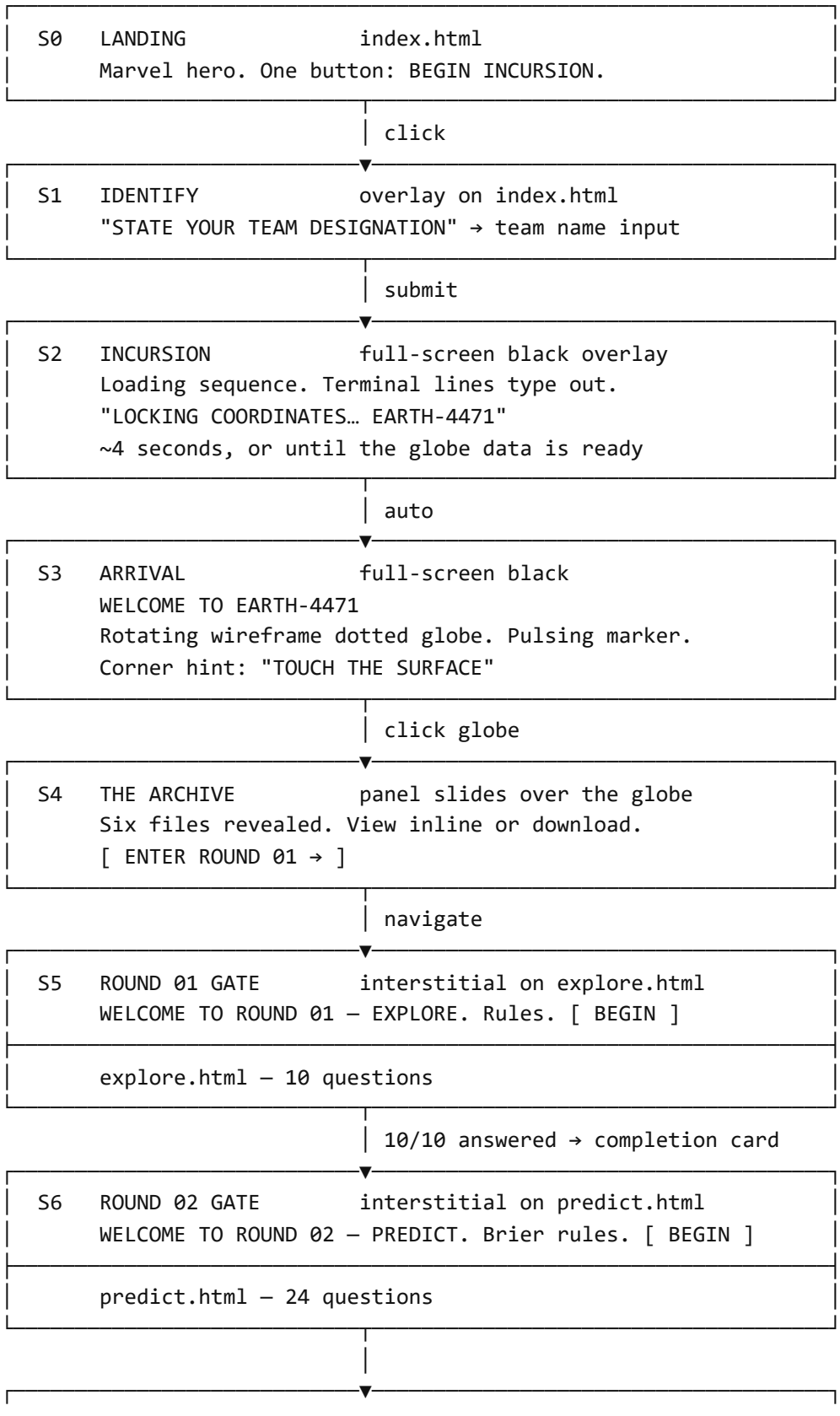
Total: ~74 KB. Small enough to load the entire archive into the browser for the in-page data browser (§7) without any pagination concerns.

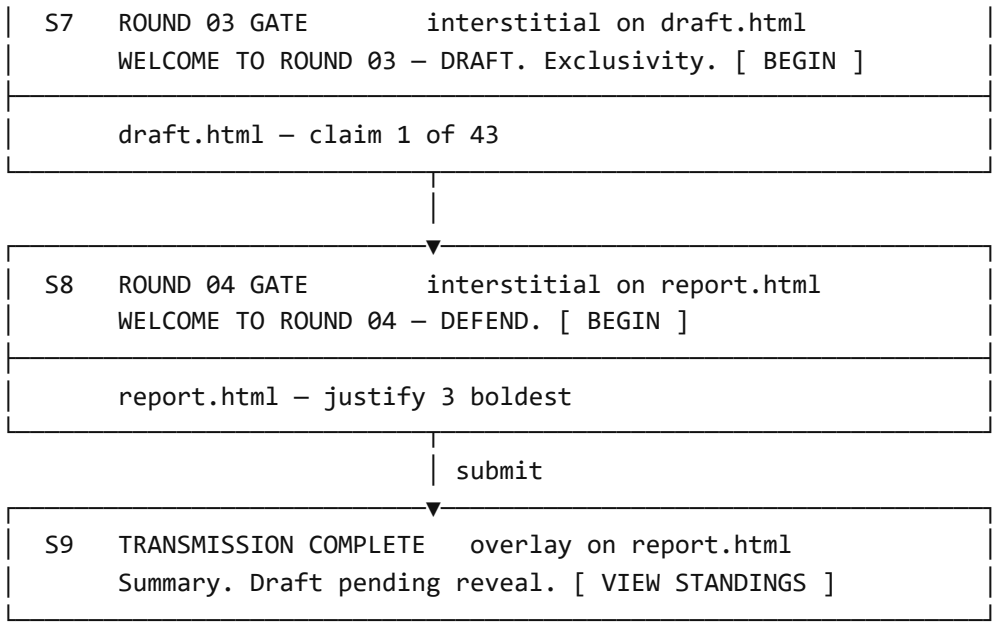
> ⚠️ **Known data flaw:** `characters.csv`'s `first_film` column is **completely empty** for all 60 rows. The generator (`entities.py`) sets it to `None` with a comment saying it would be filled once appearances exist, and nothing ever fills it. It is not load-bearing — no question depends on it, and teams can derive first appearance from `appearances.csv` themselves. **Decide before the event:** either fill it in the generator and re-seed, or drop the column from the CSV export. Shipping an empty column invites teams to waste time wondering if it means something.

---

1. THE JOURNEY

Ten screens. Four are cinematic (new), six are the game (existing, restyled).


S0 LANDING index.html
Marvel hero. One button: BEGIN INCURSION.
click
S1 IDENTIFY overlay on index.html
"STATE YOUR TEAM DESIGNATION" → team name input
submit
S2 INCURSION full-screen black overlay
Loading sequence. Terminal lines type out.
"LOCKING COORDINATES… EARTH-4471"
~4 seconds, or until the globe data is ready
auto
S3 ARRIVAL full-screen black
WELCOME TO EARTH-4471
Rotating wireframe dotted globe. Pulsing marker.
Corner hint: "TOUCH THE SURFACE"
click globe
S4 THE ARCHIVE panel slides over the globe
Six files revealed. View inline or download.
[ENTER ROUND 01 →]
navigate
S5 ROUND 01 GATE interstitial on explore.html
WELCOME TO ROUND 01 — EXPLORE. Rules. [BEGIN]
explore.html — 10 questions
10/10 answered → completion card
S6 ROUND 02 GATE interstitial on predict.html
WELCOME TO ROUND 02 — PREDICT. Brier rules. [BEGIN]
predict.html — 24 questions



Screens S1–S4 and S5–S9 gates are all overlays, not new HTML files. This is deliberate: no new routing, no new files to deploy, and — critically — **the DOM contract in `UI_DESIGN_SPEC.md` §7 stays completely untouched.** Every existing element ID keeps working because the existing markup is still there, just covered until dismissed.

---

2. DESIGN RECONCILIATION — TWO VISUAL MODES

There is a real conflict to resolve. `UI_DESIGN_SPEC.md` specifies **newsprint paper, black borders, hard shadows** — bright and printed. The globe reference is **white wireframe on pure black** — dark and digital. Bolting one onto the other looks like two different sites.

The resolution is narrative, and it makes both stronger:

> **VOID MODE is travel. ARCHIVE MODE is arrival.** The black is the space between universes. The paper is the physical archive you land in.

	VOID MODE	ARCHIVE MODE
Used by	S1, S2, S3, S4, S9	S0, S5–S8, all gameplay, admin, judge
Ground	`--ink` (#0A0A0A)	`--paper` (#F2EDE0)
Ink	`--paper` (#F2EDE0)	`--ink`
Accent	`--marvel-red` + `--glitch`	`--marvel-red` + `--marvel-yellow`
Type	`--font-mono` dominant — terminal	`--font-display` dominant — print
Borders	2px, `--paper`	3–5px, `--ink`
Shadows	Red offset (`--sh-red`)	Black offset (`--sh`)
Texture	Scanlines + starfield	Halftone dots
Feel	Cold, technical, transit	Loud, printed, permanent

The moment of transition — S4 dismissing into S5 — is the payoff: black void wipes away, paper archive slams in. Specified in §8.2.

2.1 VOID MODE TOKENS

Add to `theme.css`. Scoped, so it never leaks into ARCHIVE MODE pages.

```
.void {
  --bg: var(--ink);
  --surface: #141210;
  --text: var(--paper);
  --text-quiet: #7C7269;
  --border: var(--bw-thin) solid var(--paper);
  --sh: var(--sh-red);
  background: var(--ink);
  color: var(--paper);
  font-family: var(--font-mono);
}
.void h1, .void h2 { color: var(--paper); }
.void .win { background: var(--surface); border-color: var(--paper); box-shadow:
6px 6px 0 var(--marvel-red); }
.void .win__bar { background: var(--marvel-red); color: var(--paper-bright);
border-bottom-color: var(--paper); }
.void .win__dots i { border-color: var(--paper-bright); }
.void button { background: var(--marvel-red); color: var(--paper-bright);
border-color: var(--paper); }
.void button:hover { background: var(--paper); color: var(--ink); }
.void input { background: #0F0D0B; color: var(--paper); border-color: var(--
paper); }

/* Scanlines — the void's texture, replacing halftone */
.void::after {
  content: '';
  position: fixed; inset: 0;
  pointer-events: none;
  background: repeating-linear-gradient(
    to bottom,
    transparent 0 3px,
    rgba(255, 255, 255, 0.028) 3px 4px
  );
  z-index: 2;
}
```

> This is the **one** place the design permits a translucent value (`rgba(...,0.028)`). Scanlines are a light effect, not a surface, and a flat approximation looks wrong. Everywhere else, §2.1's no-transparency rule in `UI_DESIGN_SPEC.md` holds.

---

3. THE STATE MACHINE

One `localStorage` key drives the whole journey.

```
// portal/public/shared/journey.js

const JOURNEY_KEY = 'doomsday_journey';
const TEAM_KEY    = 'doomsday_team_id';   // existing — do not rename

const DEFAULT_STATE = {
  arrived: false,   // completed S2 incursion + S3 globe
  archived: false,  // opened the archive panel (S4)
  r1: false,        // saw the Round 1 gate
  r2: false,
  r3: false,
  r4: false,
  done: false,      // submitted the report (S9)
};

function readJourney() {
  try {
    return { ...DEFAULT_STATE, ...JSON.parse(localStorage.getItem(JOURNEY_KEY)
|| '{}') };
  } catch (e) {
    return { ...DEFAULT_STATE };   // corrupt value must never break the page
  }
}

function markJourney(key) {
  try {
    const state = readJourney();
    state[key] = true;
    localStorage.setItem(JOURNEY_KEY, JSON.stringify(state));
  } catch (e) { /* private mode / quota — journey degrades to always-show, still
playable */ }
}

function teamId() {
  try { return localStorage.getItem(TEAM_KEY); } catch (e) { return null; }
}
```

Every read is wrapped in try/catch. Safari private mode throws on `localStorage` access. A thrown exception here would blank the page for that team — unacceptable.

3.1 GATE RULES

Screen	Show when	Skip when
S1 Identify	`!teamId()`	Team ID already set
S2 Incursion	`!state.arrived`	`state.arrived`
S3 Globe	`!state.arrived`	`state.arrived`
S4 Archive	Always available via nav	—
S5–S8 Gates	`!state.rN`	`state.rN`
S9 Complete	On successful report write	—

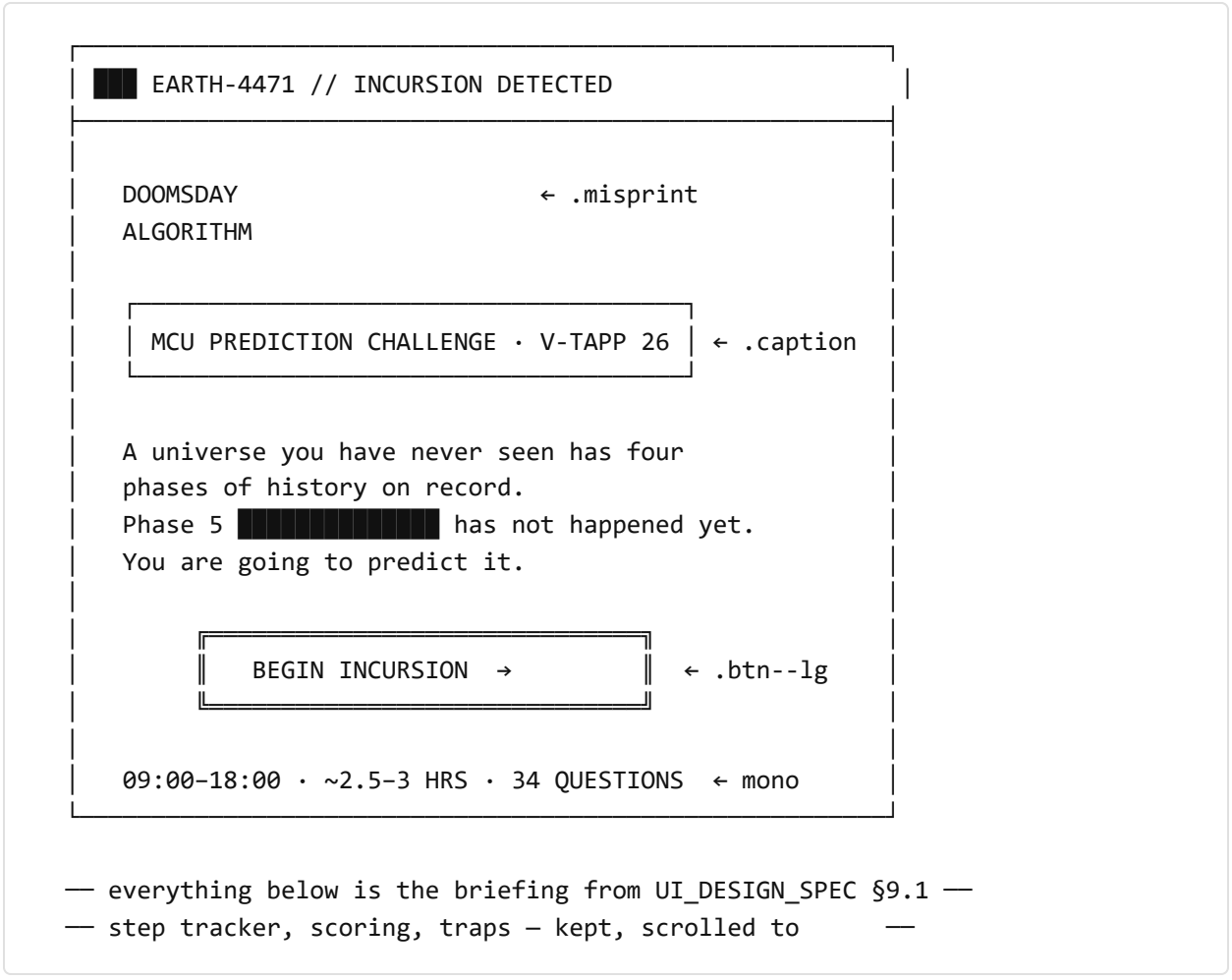
Design consequence: a team that closes the tab and returns is not made to sit through the incursion again. They land on the normal page and get straight back to work. A team that *wants* to replay the intro can, via §10.3.

---

4. SCREEN SPECIFICATIONS

S0 — LANDING (`INDEX.HTML`, ARCHIVE MODE)

The page specified in `UI_DESIGN_SPEC.md` §9.1, with **one change**: the primary call to action is now a single button.

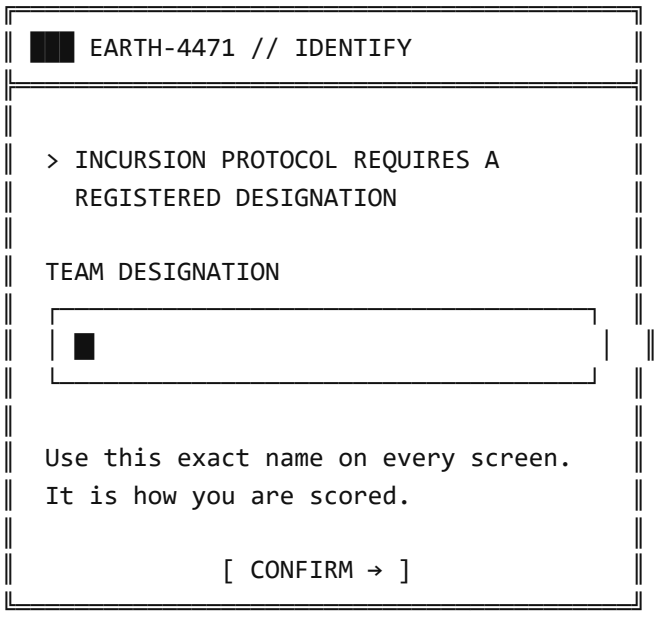


The step tracker, scoring table and traps window all remain below the fold. They are the reference material a team scrolls back to. The button is the *entry*; the briefing is the *manual*.

Returning team: if `state.arrived` is true, the button reads `RESUME →` and links directly to the first incomplete round.

S1 — IDENTIFY (OVERLAY, VOID MODE)

Triggered by `BEGIN INCURSION`. Full-screen black overlay, fades in over 200 ms.

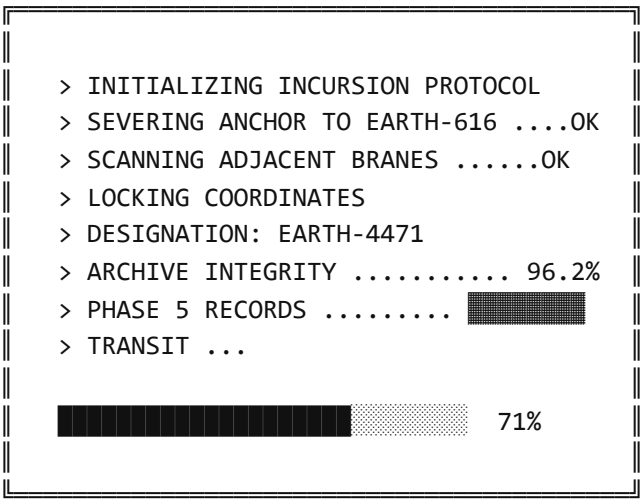


Requirements:

- Input is `--font-mono`, `--t-xl`, with a blinking block cursor (`█`) rendered as a CSS `::after` on the wrapper — pure decoration, does not interfere with the real caret.
- **Validation:** trim whitespace; require 2–24 characters; reject empty. On failure, shake the input 300 ms and show `DESIGNATION REJECTED — 2–24 CHARACTERS` in `--glitch`.
- **Normalize:** `trim()` only. **Do not uppercase or slugify** — the stored value is the Firestore document key for the leaderboard, and it must match exactly what organizers wrote down at check-in.
- On confirm: write `localStorage['doomsday_team_id']`, then advance to S2.
- **Escape hatch:** pressing `Esc` closes the overlay and returns to S0. Never trap a participant in a modal.

S2 — INCURSION (FULL-SCREEN, VOID MODE)

The loading sequence. Its real job is to cover the globe's data fetch and dot precomputation (§6.4) so S3 opens instantly instead of stuttering.

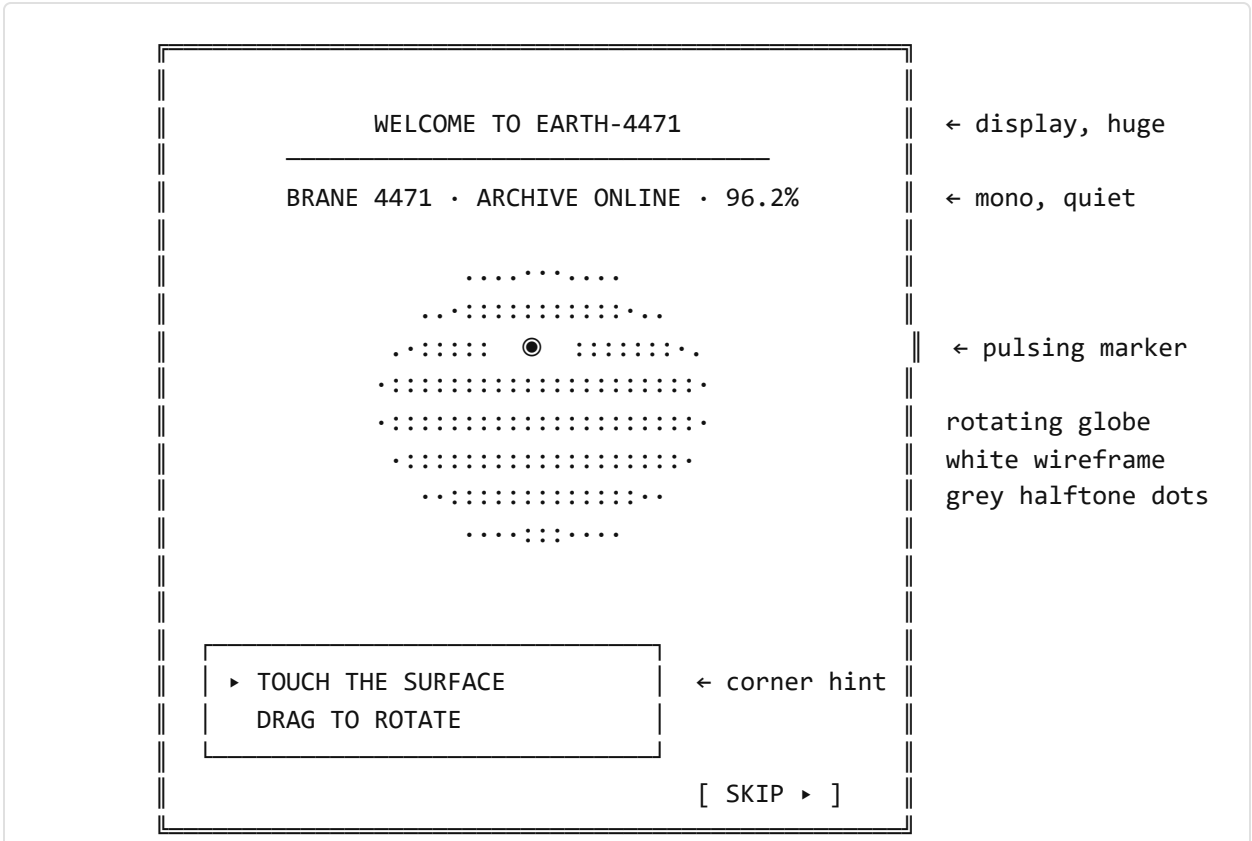


Requirements:

- Lines type out at **28 ms/character**, one after another. Full sequence ≈ 3.5 s.
- `PHASE 5 RECORDS` resolves to a `.redact` bar, not a value. This is the first hint that Phase 5 is the thing being withheld.
- `ARCHIVE INTEGRITY 96.2%` is the Earth-4471 wrongness tell. Keep it under 100.
- Progress bar is **honest where it can be**: it tracks the actual globe fetch + dot generation, and only completes when the globe is ready. If the globe finishes early, hold at 92% until the typing finishes — never let the bar finish before the text.
- **Hard timeout: 6 seconds.** If the globe is not ready by then, skip S3 entirely and jump to S4. Under no circumstances does a participant sit on a loading screen for longer than 6 seconds at an event with a queue behind them.
- **Skip control:** `[ SKIP ▸ ]` in the bottom-right, always visible, always working. Some teams will run this twice; some will not care about the cinematics. Respect that.

S3 — ARRIVAL (FULL-SCREEN, VOID MODE) — THE GLOBE

The centrepiece.



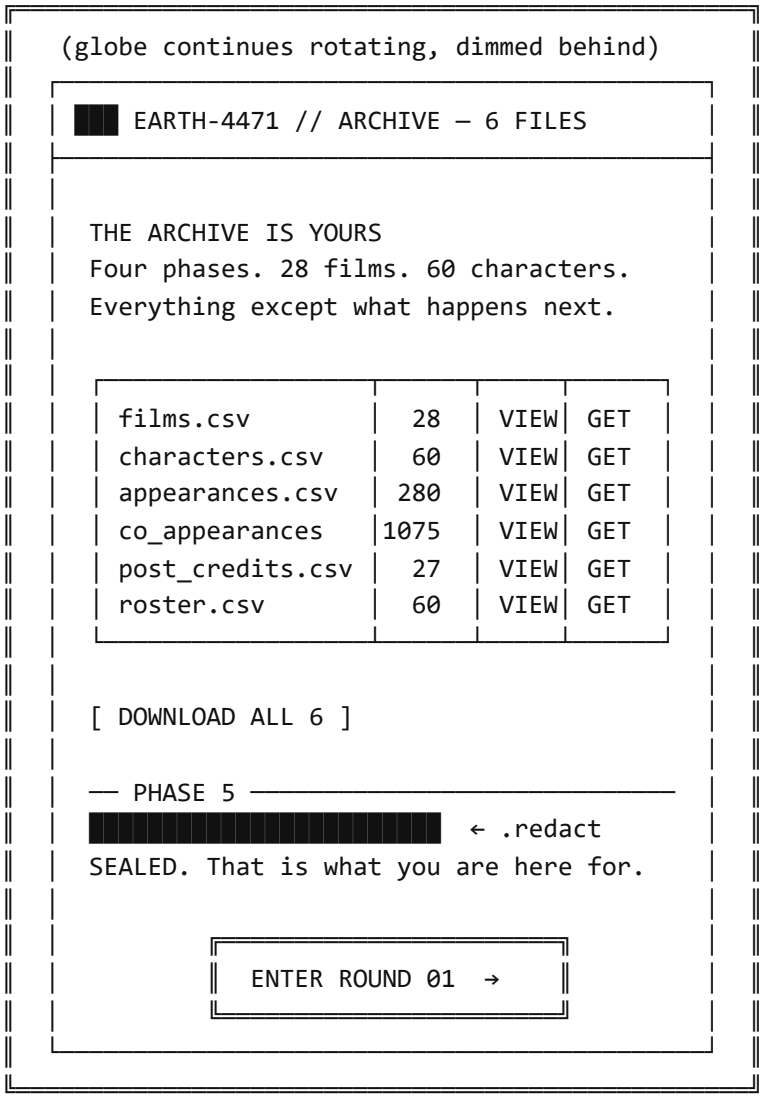
Requirements:

- Globe auto-rotates at **0.35°/frame**, capped at **30 fps** (§6.4 — this halves CPU on weak laptops and is visually indistinguishable for a slow rotation).
- Drag to rotate. Auto-rotation resumes 1.5 s after release.
- **No scroll-to-zoom.** The reference has it; we remove it. On a laptop trackpad, a two-finger scroll meant for the page instead zooms the globe, and the participant has no way to get back to the default. Not worth it.
- **The marker:** a single pulsing red dot at a fixed lat/lng, `[78, 22]` (a point in the Indian Ocean — deliberately not a real landmark, because Earth-4471 is not our Earth). Pulses 1.0→1.8× radius on a 1.6 s loop. It rotates with the globe and is hidden when it goes behind the horizon.
- **The hint (`TOUCH THE SURFACE`)** appears after 2 s. If nothing is clicked for 8 s, it starts pulsing. If nothing is clicked for 15 s, a second line appears: `▸ CLICK ANYWHERE ON THE GLOBE`.

- **Hit area is the whole globe, not the marker.** The marker is the *affordance*; the entire sphere is the *target*. Hit-testing a small rotating dot under time pressure is a frustration generator. Click anywhere within the globe radius → advance to S4.
- **Cursor:** `pointer` when over the globe, `grab` / `grabbing` while dragging.
- Skip control persists, bottom-right.

S4 — THE ARCHIVE (PANEL OVER THE GLOBE, VOID MODE)

The reveal. The globe keeps rotating behind, dimmed and pushed back.

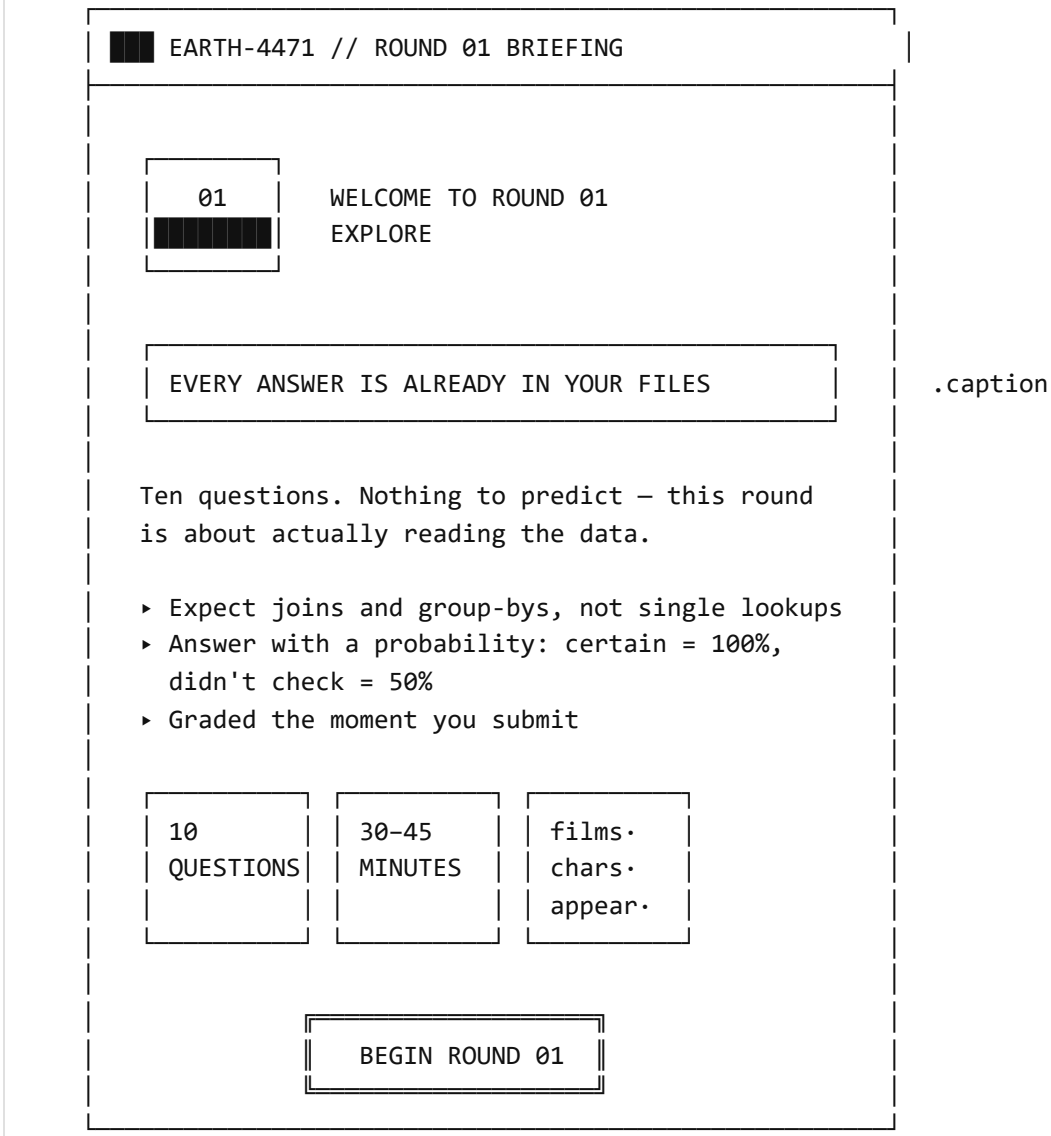


Requirements:

- Panel slides up from `translateY(24px)` + fades, 300 ms. Globe dims to 35% brightness via a black overlay beneath the panel (a solid `--ink` layer at reduced height — **not** `filter: brightness()`, which forces expensive recomposites every rotation frame).
- Each row: filename (mono), row count (mono, `--marvel-blue`), `VIEW` (opens the data browser, §7), `GET` (native download).
- `DOWNLOAD ALL 6` triggers six sequential `<a download>` clicks, 120 ms apart. Browsers block rapid multi-downloads; the stagger avoids it. Chrome will still show a “download multiple files?” prompt — that is expected and fine.
- The Phase 5 redaction bar is the emotional beat. It sits where a seventh file would be.
- `ENTER ROUND 01` sets `markJourney('archived')` and navigates to `explore.html`.
- **Reachable later:** add an `ARCHIVE` link to `nav.js`, pointing at `index.html#archive`, so teams can reopen the file list without replaying the intro.

S5–S8 — ROUND GATES (OVERLAY, ARCHIVE MODE)

One reusable component, four instances. **ARCHIVE MODE**, not void — you have arrived; these are printed briefing cards, not transit screens.



Per-round copy — use verbatim:

	R1 EXPLORE	R2 PREDICT	R3 DRAFT	R4 DEFEND
Caption	EVERY ANSWER IS ALREADY IN YOUR FILES	NO FILE CONTAINS THESE ANSWERS	ONE CHARACTER. FIRST COME, GONE FOREVER.	SHOW YOUR WORKING
Lead	Ten questions. Nothing to predict — this round is about actually reading the data.	Twenty-four questions about Phase 5, which hasn't happened. Model Phases 1–4 and extrapolate.	Claim one character you believe survives Phase 5. The moment you take them, no other team can.	We pulled your three boldest calls. Tell us what in the data backs them.
Bullet 1	Expect joins and group-bys, not single lookups	Brier scored — an honest 60% beats a bluffed 95% that's wrong	43 characters. One pick per team. Permanent.	Cite a specific chart or number, not a feeling
Bullet 2	Answer with a probability: certain = 100%, didn't check = 50%	Skipping scores the same as 50% — a real shrug costs nothing	The pool only shrinks. Earlier means more choice.	Name a trap you caught and how you handled it
Bullet 3	Graded the moment you submit	Three traps are planted. Not every correlation is real.	Scored at the 6 PM reveal, not now	Half a page. A stranger should follow it in a minute.
Stats	10 · 30–45 min · 4 files	24 · 60–90 min · Brier	43 assets · 5 min · 1 pick	3 calls · 20–30 min · hand-judged

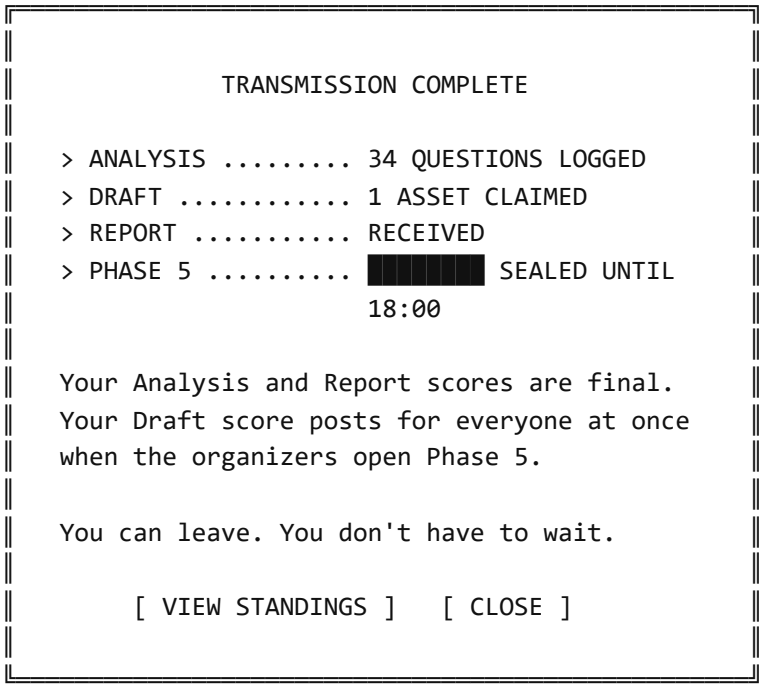
Behavior:

- Renders only if `!state.rN`. Dismiss → `markJourney('rN')` → never shown again.
- Underlying page content is present in the DOM the entire time, just covered. **The DOM contract is never violated.**
- `Esc` dismisses. So does clicking the backdrop.

- A `[ ROUND BRIEFING ]` button in the page header reopens it on demand.

S9 — TRANSMISSION COMPLETE (OVERLAY ON `REPORT.HTML`, VOID MODE)

Fires on a successful report write. Back to void — the run is over, you are leaving.



- Counts are **local** — derived from `.btn--done` counts in `sessionStorage` and the journey state. **Do not query Firestore for a summary.** (`UI_DESIGN_SPEC.md` §13.2: zero extra reads.)
- If a count is not confidently known, print `LOGGED` rather than a wrong number. Never show a number you had to guess.
- `VIEW STANDINGS` → `admin.html`. `CLOSE` → dismisses to the report page.

---

5. NEW FILE INVENTORY

Seven new files. Nothing existing is renamed.

File	~Size	Purpose
`shared/journey.js`	1 KB	State machine (§3)
`shared/overlay.js`	2 KB	Generic overlay: mount, dismiss, Esc, focus trap
`shared/gate.js`	3 KB	Round-gate content + renderer (S5–S8)
`shared/globe.js`	6 KB	Vanilla globe (§6)
`shared/intro.js`	4 KB	S1–S4 sequence orchestration
`shared/databrowser.js`	3 KB	CSV viewer (§7)
`data/land-110m.json`	~100 KB	**Vendored** Natural Earth land polygons

Plus one CDN script, on `index.html` only:

```
<script src="https://cdn.jsdelivr.net/npm/d3-geo@3/dist/d3-geo.min.js"></script>
```

Not full d3 — `d3-geo` alone is ~35 KB vs ~280 KB for the whole bundle. We use exactly three things from it: `geoOrthographic`, `geoPath`, `geoGraticule`. The reference imports all of d3 out of habit.

> **Vendor the GeoJSON. Do not fetch from `raw.githubusercontent.com` at event time.** The reference does. That is a third-party dependency on the critical path of your event's first impression, on campus wifi, from a host that rate-limits and is occasionally blocked on institutional networks. Download it once, commit it, serve it from your own origin.

---

6. THE GLOBE — VANILLA PORT

6.1 WHY THIS IS NOT A HARD PORT

The React component is **~95% imperative canvas code already**. Everything meaningful lives inside one `useEffect` that runs once. `useState` is used only for a loading flag and an error string. There is no JSX beyond a `<canvas>` and two `<div>`s.

Porting is mechanical: delete the React shell, keep the body, replace `setIsLoading` / `setError` with direct DOM updates. d3 is a plain library — it has never needed React.

6.2 WHAT WE CHANGE, AND WHY

Reference does	We do	Reason
`import * as d3 from "d3"`	`d3-geo` only, from CDN	280 KB → 35 KB
Fetches GeoJSON from GitHub	Vendored `data/land-110m.json`	No third-party on the critical path
Dot spacing `16` (~11,800 dots)	Spacing tuned to **≤4,000 dots**	3× fewer per-frame operations
Projects every dot each frame	**Back-face cull before projecting**	Skips ~50% of dots for free
`d3.timer` at ~60 fps	Throttled to **30 fps**	Halves CPU; invisible at 0.35°/frame
Scroll to zoom	**Removed**	Hijacks trackpad scrolling
`console.log('[v0] …')`	Removed	Debug noise from the generator
Grey `#999` dots	`#8A8A8A` land, `--marvel-red` marker	Marker must read at a glance
No reduced-motion handling	Static globe under `prefers-reduced-motion`	Accessibility

6.3 BACK-FACE CULLING — THE KEY OPTIMIZATION

The reference projects all ~11,800 dots every frame and lets `projection()` return `null` for hidden ones. That is thousands of wasted trig operations per frame.

Instead: precompute each dot's **3D unit vector once**, then per frame test visibility with a single dot product before projecting.

```
// Precompute ONCE per dot, at load:
const ϕ = lat * Math.PI / 180, λ = lng * Math.PI / 180;
dot.x = Math.cos(ϕ) * Math.cos(λ);
dot.y = Math.cos(ϕ) * Math.sin(λ);
dot.z = Math.sin(ϕ);

// Per frame: rotate the VIEW vector, not every dot.
const λr = -rotation[0] * Math.PI / 180, ϕr = -rotation[1] * Math.PI / 180;
const vx = Math.cos(ϕr) * Math.cos(λr);
const vy = Math.cos(ϕr) * Math.sin(λr);
const vz = Math.sin(ϕr);

// Visible iff the dot faces the camera:
if (dot.x * vx + dot.y * vy + dot.z * vz > 0) { /* project and draw */ }
```

Three multiplies and two adds to reject a hidden dot, versus a full projection. Roughly halves per-frame cost.

6.4 REFERENCE IMPLEMENTATION

```
// portal/public/shared/globe.js
// Vanilla port of the d3 wireframe-dotted-globe. Requires d3-geo on window.d3.
// Renders into an existing <canvas>. No framework.

function createGlobe(canvas, options = {}) {
  const ctx = canvas.getContext('2d');
  if (!ctx || !window.d3) return null;

  const cssW = options.width  || canvas.clientWidth  || 640;
  const cssH = options.height || canvas.clientHeight || 640;
  const dpr  = Math.min(window.devicePixelRatio || 1, 2);   // cap at 2: a 3x
                                                            // retina canvas
costs
                                                            // 9x the fill rate
for
                                                            // no visible gain
here
  canvas.width  = cssW * dpr;
  canvas.height = cssH * dpr;
  canvas.style.width  = cssW + 'px';
  canvas.style.height = cssH + 'px';
  ctx.scale(dpr, dpr);

  const radius = Math.min(cssW, cssH) / 2.4;
  const projection = window.d3.geoOrthographic()
    .scale(radius)
    .translate([cssW / 2, cssH / 2])
    .clipAngle(90);
  const path = window.d3.geoPath().projection(projection).context(ctx);
  const graticule = window.d3.geoGraticule();

  const MARKER = [78, 22];          // Earth-4471 archive site — not a real
landmark
  const MAX_DOTS = 4000;
  const FRAME_MS = 1000 / 30;       // 30 fps cap
  const REDUCED = window.matchMedia('(prefers-reduced-motion: reduce)').matches;

  let land = null, dots = [], rotation = [0, -12], autoRotate = !REDUCED;
  let rafId = null, lastFrame = 0, resumeTimer = null,

  // ---- point-in-polygon (unchanged in substance from the reference) ----
  function inRing(pt, ring) {
    const [x, y] = pt;
```

```
    let inside = false;
    for (let i = 0, j = ring.length - 1; i < ring.length; j = i++) {
      const [xi, yi] = ring[i], [xj, yj] = ring[j];
      if ((yi > y) !== (yj > y) && x < ((xj - xi) * (y - yi)) / (yj - yi) + xi)
inside = !inside;
    }
    return inside;
  }
  function inFeature(pt, feature) {
    const g = feature.geometry;
    const polys = g.type === 'Polygon' ? [g.coordinates]
                : g.type === 'MultiPolygon' ? g.coordinates : [];
    for (const poly of polys) {
      if (!inRing(pt, poly[0])) continue;
      let hole = false;
      for (let i = 1; i < poly.length; i++) if (inRing(pt, poly[i])) { hole =
true; break; }
      if (!hole) return true;
    }
    return false;
  }

  // ---- dot generation, with a hard budget ----
  // Step size is derived from the target count rather than hard-coded, so the
  // globe costs the same on every machine regardless of how much land the
  // GeoJSON happens to contain.
  function buildDots(step) {
    const out = [];
    for (const feature of land.features) {
      const [[minLng, minLat], [maxLng, maxLat]] = window.d3.geoBounds(feature);
      for (let lng = minLng; lng <= maxLng; lng += step) {
        for (let lat = minLat; lat <= maxLat; lat += step) {
          if (inFeature([lng, lat], feature)) {
            const p = lat * Math.PI / 180, l = lng * Math.PI / 180;
            out.push({
              lng, lat,
              x: Math.cos(p) * Math.cos(l),
              y: Math.cos(p) * Math.sin(l),
              z: Math.sin(p),
            });
          }
        }
      }
    }
    return out;
  }
```

```
  function generateDots() {
    let step = 1.6;
    dots = buildDots(step);
    // Coarsen until under budget. Converges in 1-3 passes for Natural Earth
110m.
    let guard = 0;
    while (dots.length > MAX_DOTS && guard++ < 4) {
      step *= 1.25;
      dots = buildDots(step);
    }
  }

  function viewVector() {
    const l = -rotation[0] * Math.PI / 180, p = -rotation[1] * Math.PI / 180;
    return [Math.cos(p) * Math.cos(l), Math.cos(p) * Math.sin(l), Math.sin(p)];
  }

  function render(t) {
    ctx.clearRect(0, 0, cssW, cssH);

    // Sphere
    ctx.beginPath();
    ctx.arc(cssW / 2, cssH / 2, radius, 0, 2 * Math.PI);
    ctx.fillStyle = '#050505';
    ctx.fill();
    ctx.strokeStyle = '#F2EDE0';
    ctx.lineWidth = 2;
    ctx.stroke();

    if (!land) return;

    // Graticule — drawn faintly via a light stroke color, NOT globalAlpha,
    // so we never touch compositing state mid-frame.
    ctx.beginPath();
    path(graticule());
    ctx.strokeStyle = '#3A3733';
    ctx.lineWidth = 1;
    ctx.stroke();

    // Coastlines
    ctx.beginPath();
    for (const f of land.features) path(f);
    ctx.strokeStyle = '#F2EDE0';
    ctx.lineWidth = 1;
    ctx.stroke();
```

```
  // Halftone land dots, back-face culled
  const [vx, vy, vz] = viewVector();
  ctx.fillStyle = '#8A8A8A';
  for (let i = 0; i < dots.length; i++) {
    const d = dots[i];
    if (d.x * vx + d.y * vy + d.z * vz <= 0) continue;
    const p = projection([d.lng, d.lat]);
    if (!p) continue;
    ctx.beginPath();
    ctx.arc(p[0], p[1], 1.25, 0, 2 * Math.PI);
    ctx.fill();
  }

  // Archive marker — pulsing, hidden when behind the horizon
  const m = projection(MARKER);
  const mp = MARKER[1] * Math.PI / 180, ml = MARKER[0] * Math.PI / 180;
  const facing = Math.cos(mp) * Math.cos(ml) * vx
               + Math.cos(mp) * Math.sin(ml) * vy
               + Math.sin(mp) * vz;
  if (m && facing > 0) {
    const pulse = 1 + 0.8 * (0.5 + 0.5 * Math.sin(t / 800));
    ctx.beginPath();
    ctx.arc(m[0], m[1], 4.5 * pulse, 0, 2 * Math.PI);
    ctx.fillStyle = '#ED1D24';
    ctx.fill();
    ctx.beginPath();
    ctx.arc(m[0], m[1], 9 * pulse, 0, 2 * Math.PI);
    ctx.strokeStyle = '#ED1D24';
    ctx.lineWidth = 1.5;
    ctx.stroke();
  }
}

function frame(t) {
  rafId = requestAnimationFrame(frame);
  if (t - lastFrame < FRAME_MS) return;      // 30 fps throttle
  lastFrame = t;
  if (autoRotate) {
    rotation[0] += 0.35;
    projection.rotate(rotation);
  }
  render(t);
}

// ---- interaction ----
```

```
function isOnGlobe(e) {
  const r = canvas.getBoundingClientRect();
  const dx = e.clientX - r.left - cssW / 2;
  const dy = e.clientY - r.top - cssH / 2;
  return dx * dx + dy * dy <= radius * radius;
}

let dragging = false, moved = 0;
function down(e) {
  if (!isOnGlobe(e)) return;
  dragging = true; moved = 0; autoRotate = false;
  clearTimeout(resumeTimer);
  const sx = e.clientX, sy = e.clientY, start = rotation.slice();
  canvas.style.cursor = 'grabbing';

  function move(ev) {
    const dx = ev.clientX - sx, dy = ev.clientY - sy;
    moved = Math.max(moved, Math.abs(dx) + Math.abs(dy));
    rotation[0] = start[0] + dx * 0.4;
    rotation[1] = Math.max(-80, Math.min(80, start[1] - dy * 0.4));
    projection.rotate(rotation);
  }
  function up(ev) {
    document.removeEventListener('mousemove', move);
    document.removeEventListener('mouseup', up);
    dragging = false;
    canvas.style.cursor = 'pointer';
    resumeTimer = setTimeout(() => { autoRotate = !REDUCED; }, 1500);
    // A drag is not a click. 6px of slop tolerates trackpad jitter.
    if (moved < 6 && onClick && isOnGlobe(ev)) onClick();
  }
  document.addEventListener('mousemove', move);
  document.addEventListener('mouseup', up);
}

canvas.addEventListener('mousedown', down);
canvas.style.cursor = 'pointer';

return {
  async load() {
    const res = await fetch('data/land-110m.json');
    if (!res.ok) throw new Error('archive map unavailable');
    land = await res.json();
    generateDots();
    projection.rotate(rotation);
    rafId = requestAnimationFrame(frame);
```

```
      return dots.length;
    },
    onSurfaceClick(fn) { },
    destroy() {
      if (rafId) cancelAnimationFrame(rafId);
      clearTimeout(resumeTimer);
      canvas.removeEventListener('mousedown', down);
    },
  };
}
```

6.5 TOUCH SUPPORT

The reference is mouse-only. Some teams will be on tablets. Add:

```
canvas.addEventListener('touchstart', e => {
  if (e.touches.length !== 1) return;
  down({ clientX: e.touches[0].clientX, clientY: e.touches[0].clientY });
}, { passive: true });
```

with matching `touchmove` / `touchend` handlers mapping `e.touches[0]` / `e.changedTouches[0]`. **Do not** `preventDefault` on `touchmove` outside the globe radius, or the participant cannot scroll the page.

6.6 FAILURE HANDLING — NON-NEGOTIABLE

```
try {
  const globe = createGlobe(canvasEl);
  if (!globe) throw new Error('canvas unavailable');
  await globe.load();
} catch (err) {
  skipToArchive();     // straight to S4, no globe, no error dialog
}
```

Any globe failure — no canvas, no d3, GeoJSON 404, slow device — skips silently to S4. The participant never sees an error about a decorative element. Log to `console.warn` for the organizer; show the participant nothing.

---

7. THE DATA BROWSER

`VIEW` on any archive row opens the file inline. Some teams arrive without Excel; some just want a look before committing to a download.

```
┌──────────────────────────────────────────────────────────┐
│ ▓▓▓ EARTH-4471 // FILE: FILMS.CSV — 28 ROWS               │
├──────────────────────────────────────────────────────────┤
│  [ filter rows…            ]        [ GET FILE ] [ × ]  │
├──────────────────────────────────────────────────────────┤
│ name               │phase│year│budget│gross │critic│aud  │
│════════════════════╪═════╪════╪══════╪══════╪══════╪═════│
│ E-4471 Chronicle 1 │  1  │2019│168.4 │360.9 │  59  │ 53  │
│ E-4471 Chronicle 2 │  1  │2019│171.2 │388.1 │  74  │ 61  │
│ …                                                        │
├──────────────────────────────────────────────────────────┤
│ SHOWING 28 OF 28 ROWS                                    │
└──────────────────────────────────────────────────────────┘
```

Requirements:

- Table uses `.board` styling from `UI_DESIGN_SPEC.md` §6.8, in **VOID MODE** colors.
- **Show every row.** The largest file is 1,075 rows — trivial for the DOM, and pagination would only get in the way of a team scanning for something.
- Scroll container: `max-height: 60vh; overflow: auto`, with `position: sticky; top: 0` on `<thead>`.
- Filter box does a case-insensitive substring match across the whole row. Debounce 150 ms.
- **CSV parsing:** the generated files contain no quoted fields, no embedded commas, no embedded newlines — every value is a name, a number, or a boolean. A plain `split(',')` is correct here.

```
function parseCSV(text) {
  const lines = text.trim().split(/\r?\n/);
  const header = lines[0].split(',');
  return { header, rows: lines.slice(1).map(l => l.split(',')) };
}
```

> Do not reach for a CSV library. If the dataset generator ever emits quoted fields, this breaks loudly and obviously in testing — which is the right failure mode for a file you control.

- **Escape every cell.** The data is generator-produced and safe today, but the browser renders it into the DOM; use `textContent`, never `innerHTML`, per cell.
- Fetch each CSV once, cache in memory. Six files, 74 KB total.

---

8. TRANSITIONS

8.1 WITHIN VOID MODE (S1 → S2 → S3 → S4)

From → To	Motion
S0 → S1	Black overlay fades in 200 ms; panel scales 0.96→1
S1 → S2	Panel fades out 150 ms; terminal lines begin typing
S2 → S3	Terminal text fades 300 ms; globe canvas fades in 400 ms; title drops from `-20px`
S3 → S4	Panel slides `+24px` →0 with fade, 300 ms; dim layer fades in beneath it

8.2 THE MODE CHANGE (S4 → S5) — THE PAYOFF MOMENT

The single most important transition on the site. Void becomes archive.

```
── 0 ms ──── click ENTER ROUND 01
── 0–180 ms — black wipe sweeps left→right across the viewport
              (a solid --ink panel, transform: scaleX(0)→1,
               transform-origin: left)
── 180 ms ─── navigate to explore.html
── on load ── paper page renders behind a full-screen --ink layer
── 0–200 ms — that layer wipes away left→right (scaleX(1)→0,
               transform-origin: right)
── 200 ms ─── Round 01 gate card slams in:
               scale(1.04)→1 + shadow 0→10px, 160 ms, no bounce
```

The wipe direction is continuous across the navigation — out to the right, in from the left — so it reads as one motion across a page load rather than two unrelated animations.

Implementation note: the outgoing wipe must complete before `location.href` is set, or the browser may discard the animation. Use a `setTimeout` matched to the transition duration, and set a `sessionStorage` flag the destination page reads to know it should play the incoming wipe.

```
sessionStorage.setItem('doomsday_wipe', '1');
```

The destination checks and clears it on load. If absent (direct navigation, refresh), no wipe plays — correct behavior.

8.3 REDUCED MOTION

Under `prefers-reduced-motion: reduce`: no wipes, no typing animation (all terminal lines appear at once), globe renders static at its initial rotation. **All content and every control remains reachable.** The journey still works; it just does not move.

---

9. NAV CHANGES

`nav.js` gains one entry and a progress indicator.

```
const PAGES = [
  { id: 'landing',     label: 'Home',        href: 'index.html' },
  { id: 'archive',     label: 'Archive',     href: 'index.html#archive' },   //
NEW
  { id: 'explore',     label: 'Round 1',     href: 'explore.html' },         //
renamed
  { id: 'predict',     label: 'Round 2',     href: 'predict.html' },         //
renamed
  { id: 'draft',       label: 'Round 3',     href: 'draft.html' },           //
renamed
  { id: 'report',      label: 'Round 4',     href: 'report.html' },          //
renamed
  { id: 'leaderboard', label: 'Standings',   href: 'admin.html' },
];
```

Round numbers in the nav, not round names. The journey is sequential now; the numbers communicate that at a glance and match the gate cards.

Each visited round link gets a small filled square before its label, driven by journey state — a progress trail:

```
nav.top-nav a.visited::before { content: '▪ '; color: var(--marvel-green); }
```

Purely local state. **No Firestore reads.**

---

10. RESUME, SKIP, AND FAILURE

10.1 RETURNING MID-EVENT

A team closes the laptop and comes back an hour later. On any page load:

1. `teamId()` exists → no identify prompt.
2. `state.arrived` is true → no incursion, no globe.
3. `state.rN` is true → no gate for that round.

They land exactly where they left off, with zero ceremony. **The cinematics are a first-run experience, not a toll booth.**

10.2 NEVER-BLOCKING RULE

Every overlay in this document must be dismissible by:

- its own button, **and**
- the `Esc` key, **and**
- a backdrop click (except S1, which needs a value)

An overlay that can trap a participant is a bug, not a design choice.

10.3 REPLAYING THE INTRO

For organizers demoing the event, and for teams who want to see it again:

```
https://doomsdayalgorithm-5f1e8.web.app/?replay=1
```

Clears `doomsday_journey` (leaving the team ID intact) and runs the full sequence.

10.4 THE KILL SWITCH — REQUIRED

```
https://doomsdayalgorithm-5f1e8.web.app/?plain=1
```

Skips **all** cinematics permanently for that browser: no incursion, no globe, no gates. Straight to functional pages.

Additionally, the intro **auto-disables itself** when:

- `d3` fails to load within 3 seconds
- `data/land-110m.json` returns non-200
- any exception is thrown during intro setup (wrap the whole orchestration in one `try/catch`)
- `navigator.hardwareConcurrency <= 2` (very low-end device — skip the globe, keep the gates)

> **Print `?plain=1` on the organizer's run sheet.** If anything about the intro misbehaves during the event, the fix is one URL, not a debugging session in front of a queue of students.

---

11. IMPLEMENTATION ORDER — THE FALLBACK LADDER

Build in this order. Every level ships on its own. Do not start a level you cannot finish.

LEVEL 0 — BASELINE (ALREADY DONE, ALREADY DEPLOYED)

The working game: 7 pages, 34 questions, live grading, verified end-to-end. **If everything below is abandoned, the event still runs.**

LEVEL 1 — VISUAL IDENTITY ★ HIGHEST VALUE PER HOUR

`UI_DESIGN_SPEC.md` §14 steps 1–4: tokens, chrome, component library, landing page rebuild. No JS risk, no new state, transforms how the site feels. **If you do only one level, do this one.**

LEVEL 2 — ROUND GATES (S5–S8)

`journey.js` + `overlay.js` + `gate.js`. Delivers "WELCOME TO ROUND N + instructions" — the core of what was asked for — with no globe, no d3, no external assets. Pure DOM overlays over pages that already work.

LEVEL 3 — IDENTIFY + INCURSION (S1, S2)

Team-name capture and the terminal loading sequence. Text and CSS only. Gives the cinematic on-ramp without the heaviest dependency.

LEVEL 4 — THE ARCHIVE PANEL + DATA BROWSER (S4, §7)

File reveal with inline CSV viewing. Genuinely useful to participants, independent of the globe.

LEVEL 5 — THE GLOBE (S3, §6) ★ HIGHEST COST, HIGHEST SPECTACLE

d3-geo, vendored GeoJSON, canvas rendering, dot generation, culling, touch. **Test on the actual event laptops before committing to it.** If it stutters, Level 4's archive panel stands in perfectly — the sequence just goes incursion → archive.

LEVEL 6 — POLISH

Wipe transitions (§8.2), S9 completion screen, nav progress trail, headline word cycling.

Time check: if it is the night before and Level 2 is not done, stop. Ship Level 1 and go to sleep. A well-styled, reliable site beats a half-wired cinematic that fails in front of forty students.

---

12. RISK REGISTER

Risk	Likelihood	Impact	Mitigation
Globe stutters on low-end laptops	**High**	Medium	30 fps cap, ≤4000 dots, back-face culling, `hardwareConcurrency` auto-skip, `?plain=1`
d3 CDN blocked on campus network	Medium	Medium	3 s timeout → auto-skip to S4. Optionally vendor `d3-geo.min.js` too (35 KB)
GeoJSON fetch fails	Low	Medium	Vendored locally; failure skips to S4 silently
Team ID typo'd at S1	**High**	**High**	Organizers assign IDs at check-in; input is trimmed but **never** transformed
`localStorage` throws (private mode)	Low	High	Every access wrapped in try/catch; degrades to always-show, still fully playable
Overlay traps a participant	Low	**High**	§10.2 — three independent dismiss paths, mandatory
Gate shown every load (state not saving)	Medium	Low	Annoying, not blocking. Gates are dismissible.
Redesign breaks a DOM contract ID	Medium	**Critical**	`UI_DESIGN_SPEC.md` §7; overlays never modify underlying markup
Wipe animation leaves a black screen	Low	**Critical**	Incoming wipe element gets a 1 s CSS-animation failsafe that removes it regardless of JS
Intro adds Firestore reads	Low	High	Journey state is 100% `localStorage`; data browser reads static CSVs

The two Criticals both have the same character: they fail *silently and late*. Test both explicitly against the live deployment, not locally.

---

13. ACCEPTANCE CHECKLIST

Journey:

- ☐ First visit: landing → identify → incursion → globe → archive → Round 1 gate, uninterrupted
- ☐ Second visit: lands directly on the page, zero cinematics
- ☐ `?replay=1` restores the full sequence; `?plain=1` disables it permanently
- ☐ Every overlay dismissible by button, `Esc`, and backdrop (except S1's backdrop)
- ☐ Team ID stored **exactly** as typed — no case change, no slugification
- ☐ Each round gate appears once, then never again
- ☐ Report submit triggers S9

Globe:

- ☐ Loads in under 2 s on the event laptop
- ☐ Holds ≥25 fps while rotating
- ☐ Drag rotates; release resumes after 1.5 s
- ☐ Click anywhere on the sphere opens the archive; a drag does **not** count as a click
- ☐ Marker hides behind the horizon
- ☐ Killing the network mid-load skips to S4 with no visible error
- ☐ `prefers-reduced-motion` renders it static
- ☐ No scroll hijacking anywhere on the page

Integrity:

- ☐ Zero new Firestore reads (verify in the Firebase console usage graph)
- ☐ Every `UI_DESIGN_SPEC.md` §7 ID still present and functional
- ☐ All 7 pages load with no console errors
- ☐ Submitting one question on each round still produces a `leaderboard` row

- ☐ `judge.html` still unlinked from nav
- ☐ Total added page weight under 200 KB (d3-geo 35 + GeoJSON 100 + own JS ~20)

---

14. WHAT I WOULD CUT IF ASKED

Honest priorities, best value first:

1. **Round gates (Level 2)** — this is the actual request. “Welcome to Round 1, here are the instructions” is what makes the event feel structured rather than like a form. Cheapest thing here, highest impact.
2. **Visual identity (Level 1)** — turns a functional site into an event.
3. **Archive panel (Level 4)** — the dataset reveal is a real moment, and inline CSV viewing genuinely helps teams without Excel.
4. **Incursion loader (Level 3)** — good atmosphere, pure CSS, low risk.
5. **The globe (Level 5)** — the most impressive and the most likely to misbehave on a borrowed laptop ten minutes before the event. **Build it last, and only if Levels 1–4 are done and tested.**

The globe is the thing most worth wanting and the thing least worth risking the event over. That ordering is deliberate.